

Základy syntaxe C

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2023

Základy strukturovaného programování, ZSP
10/2023, Lekce 2

<https://moodle.vspj.cz/course/view.php?id=203350>



Data a výrazy

- Programy pracují s informací.
- Ta musí být programu přístupná – musí být uložena v paměti počítače.
- V imperativních jazycích k tomu máme konstanty a proměnné.
- Způsob uložení informace závisí na počítači, operačním systému a programovacím jazyce.
- Způsob uložení triviálních informací není v C (i C++) určen striktně, jsou jen stanovena určitá pravidla.
- Zápis triviálních konstantních informací v programu nazýváme literály.
- Ten je (na rozdíl od reprezentace) určen striktně.

Popis jazyka

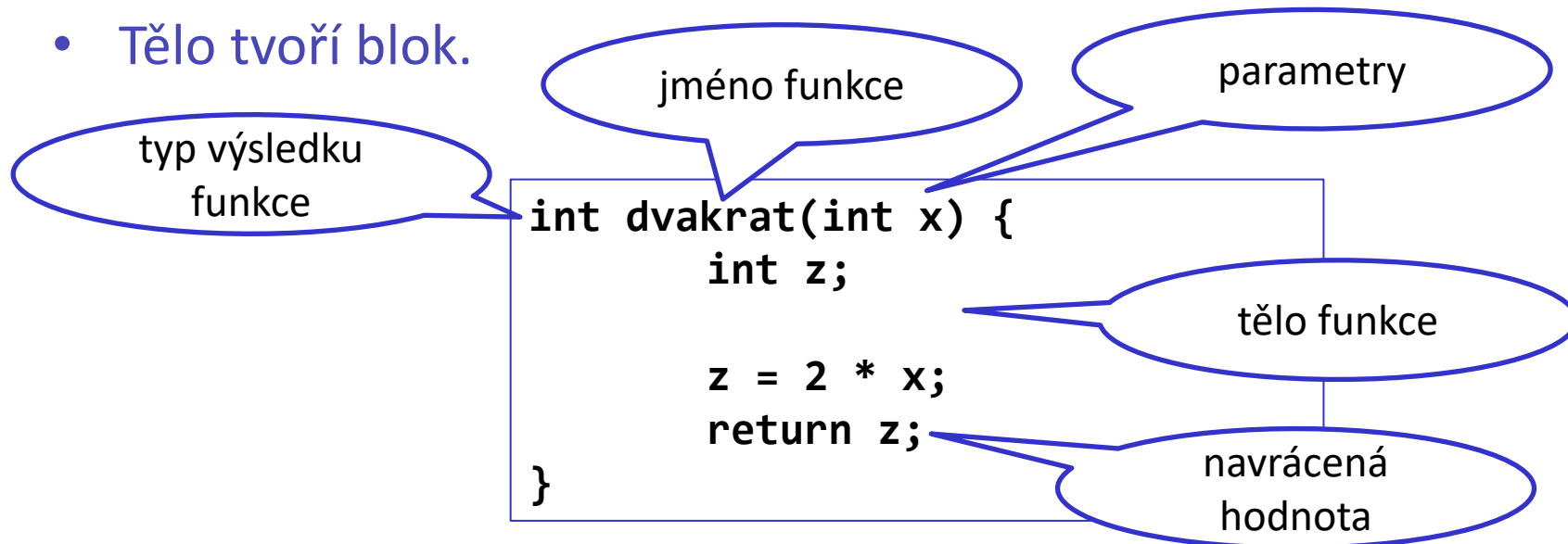
- Popis syntaxe (**jak** se to píše)
- Popis sémantiky (**co** to znamená)
- Standardní knihovny (standardní služby, které musí každá implementace jazyka poskytovat – základ přenositelnosti programů)

Popis syntaxe a sémantiky je uspořádán takto:

- popis použité paměti (**deklarace**)
- popis výpočtů prováděných nad pamětí (**výrazy**)
- popis posloupnosti provádění výpočtů (**příkazy**)

Jak se to píše?

- Program v C je sada funkcí (právě jedna se jmenuje **main**).
- Definice funkce se skládá z hlavičky a těla.
- Hlavička popisuje jak se funkce volá, tělo, jak se počítá.
- Hlavička definuje jméno funkce, parametry a výstup.
- Tělo tvoří blok.



Lexikální elementy (písmena jazyka C)

- **identifikátory**

písmena (rozlišuje se velikost – case sensitive), číslice, '_'
délka není omezena, rozlišují se podle úvodních znaků v závislosti na implementaci (ANSI doporučuje rozlišovat podle prvních 31 znaků)
některé jsou rezervovány jako klíčová slova

- **klíčová slova**

if while ... malými písmeny!

- **omezovače a operátory**

{ } ; + - & && < <=

- **číselné literály (zápisy čísel)**

125 125.45e-7 0.12E3

oktalové: 037 hexadecimální: 0x1F

Lexikální elementy (pokračování)

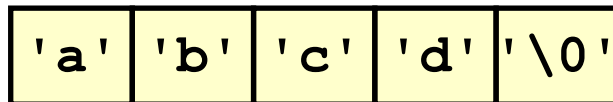
- **znakové literály (zápisy znaků)**

`'x' '\n' '\t' '\0' '\\' '\"' '\014'`

- **řetězce**

`"abcd" "" "xyz\nabc"`

vnitřní reprezentace řetězců: posloupnost znaků zakončená binární nulou `'\0'`



- **komentáře**

`/* toto je komentář */`

`// toto je komentář v novější normě C a v C++`

Typy dat

Jednoduché typy

- celočíselné (pevná řádová čárka)
- racionální (pohyblivá řádová čárka)
- **void**: typ s prázdnou množinou hodnot

Odvozené (strukturované) typy

- pole
- struktura (záznam), třída
- union (sjednocení)

Funkce

Ukazatelé

Jednoduché datové typy

- ***celočíselné typy:***

- `char` (nejmenší adresovatelná buňka)

- `char`, `unsigned char`, `signed char`

- `int` (šířka je závislá na procesoru a systému)

- `short` = `short int` = `signed short` = `signed short int`

- `int` = `signed` = `signed int`

- `unsigned` = `unsigned int`

- `long` = `long int` = `signed long` = `signed long int`

- `unsigned long` = `unsigned long int`

- `long long` = `long long int`

- `unsigned long long` = `unsigned long long int`

- `short` <= `int` <= `long`

- ***s pohyblivou řádovou čárkou:***

- `float`, `double`, `long double`

- ***logické hodnoty (v C chybí, jsou v C++):***

- 0 – nepravda, 1 – pravda (obecně cokoliv nenulového)

- ***void*** – typ s prázdnou množinou hodnot

Jednoduché datové typy: příklad reprezentace

Vnitřní reprezentace skalárních typů na PC

- celočíselné typy (reprezentace v pevné řádové čárce) v doplňkovém kódu se znaménkem
- reálné typy (reprezentace v pohyblivé řádové čárce) podle normy IEC 60559:1989 (ANSI/IEEE)

Velikost vnitřní reprezentace:	Clang++3.3, 64bit Linux	VC++ 12 64bit
char, signed char, unsigned char	1B	1B
short, unsigned short	2B	2B
int, unsigned int	4B	4B
long, unsigned long	8B	4B
long long, unsigned long long	8B	8B
float	4B	4B
double	8B	8B
long double	16B	8B
<i>Platforma</i>	<i>I32LP64</i>	<i>IL32P64</i>

I32LP64: Integer má 32 bitů, Long a Pointer mají 64 bitů

Cvičení: Jak to zjistit?

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("Reprezentace zakladnich typu dat:\n");
    printf("Delka reprezentace char = %d\n",sizeof(char));
    printf("Delka reprezentace int = %d\n",sizeof(int));
    printf("Delka reprezentace float = %d\n",sizeof(float));
    ...

    printf("Delka reprezentace ukazatelu = %d\n",
           sizeof(void *));
    return;
}
```



Odvozené (strukturnované) typy

- **Pole**

jednorozměrná pole prvků libovolného, ale stejného typu (kromě funkce – prvkem pole nemůže být funkce) – **homogenní** struktura

indexováno vždy od 0

vícerozměrná pole nejsou – musí se simulovat jako pole polí

- **Struktura (záznam), třída**

obsahuje pojmenované položky různých typů uložené za sebou – **heterogenní** struktura

- **Union (sjednocení)**

pojmenované položky různých typů uložené "přes" sebe

Funkce a ukazatelé

- **Funkce**

specifikuje funkci typem návratové hodnoty a typy parametrů

- **Ukazatel**

adresa datového objektu (proměnné nebo konstanty) nebo funkce specifikovaného typu

- **Klasifikace typů:**

- aritmetické typy: celočíselné + racionální
- skalární typy: aritmetické + ukazatelé

Deklarace proměnné

paměťová třída kvalifikátor specifikace typu seznam deklarátorů ;

Příklady:

int n;

const int x = 2;

extern pole[10];

static long double z;

Deklarace proměnné: paměťová třída

paměťová třída ***kvalifikátor*** ***specifikace typu*** ***seznam deklarátorů ;***

Paměťová třída popisuje způsob přidělování paměti:

(*nic*) dle kontextu (viz. níže)

static paměť přidělena staticky během překladu

extern nepřidělovat paměť

- Implicitně je paměťová třída:

automatická (ve funkcích, v bloku) nebo

static (v modulu, globální proměnné)

- Automatické proměnné jsou alokovány na zásobníku, statické zpravidla v datovém segmentu programu. Statické existují po celou dobu výpočtu, automatické jen po dobu zpracování bloku, ve kterém jsou lokální.

Deklarace proměnné: kvalifikátor

paměťová třída **kvalifikátor** *specifikace typu* *seznam deklarátorů* ;

Kvalifikátor popisuje další požadované vlastnosti:

const	konstantní objekt
volatile	objekt, který je sdílen s jiným nezávislým procesem
mutable	vzácné, třídní proměnné které mohou být modifikovány na const objektu

extern const int Max;

...

Max = 10; /* chyba */

volatile int regs;

Deklarace proměnné: specifikace typu

paměťová třída *kvalifikátor* **specifikace typu** *seznam deklarátorů* ;

Specifikace typu je:

- **void** zastupuje „žádný“ typ (má prázdnou množinu hodnot)
- *identifikátor typu* (základního nebo uživatelem definovaného)
- popis struktury (**struct**), popis třídy (**class**), popis sjednocení (**union**), nebo výčtového typu (**enum**)
- **auto** typ je doplněn kompilátorem jako typ na pravé straně výrazu

Deklarace proměnné: seznam deklarátorů

*paměťová třída kvalifikátor specifikace typu **seznam deklarátorů** ;*

Seznam deklarátorů je výčet deklarátorů oddělený čárkami.

Deklarátor má syntaxi:

identifikátor

* deklarátor

ukazatel

deklarátor [konstantní výraz]

pole

deklarátor (parametry)

funkce

(deklarátor)

Pozn.: za * může být kvalifikátor.

Seznam deklarátorů

paměťová třída *kvalifikátor* *specifikace typu* **seznam deklarátorů** ;

Příklady:

`int i;`

proměnná typu `int`

`int *ai;`

proměnná typu ukazatel na `int`

`int ia[10];`

proměnná typu pole 10-ti prvků typu
`int`

`int fi(double);`

proměnná typu funkce z `double` do
`int`

`(int i);`

proměnná typu `int`

Cvičení: Funkce, která si počítá volání

```
int count()  
{  
    ?  
}
```

Čtení složitějších deklarací

Deklarátory je třeba číst "**zevnitř ven**" postupně podle následujících pravidel:

- najdi deklarovaný identifikátor a zjisti, zda se **vpravo** od něj nachází [nebo (
- interpretuj tyto závorky a pak zjisti, zda se **vlevo** od identifikátoru nachází *
- kdykoliv narazíš na), vrať se zpět a aplikuj předchozí pravidla uvnitř závorek (a)
- nakonec použij specifikaci typu

Příklad čtení deklarace

```
char * ( * (*x) () ) [10] ;
```

- 1.identifikátor **x** je deklarován jako
- 2.ukazatel na
- 3.funkci vracející
- 4.ukazatel na
- 5.pole 10-ti elementů, kterými jsou
- 6.ukazatelé na
- 7.hodnoty typu **char**

Definice vlastních typů

Syntaxe:

```
using alias = typ;
```

```
typedef typ alias
```

Příklady:

```
using Byte = unsigned char;  
typedef unsigned char Byte;
```

```
using Word = short int;  
typedef unsigned short int Word;
```

```
using Matice = int[10][10];  
typedef int Matice[10][10];
```

Možné deklarace:

```
static Word *ptr;  
const Matice m = {{ /**/ }};
```

```
const Byte b = 0xBE;  
extern Word w;
```

Nepovolené typy

- **void** mimo následující povolené možnosti:

`void f();`

`... f(void);`

`void *`

- pole funkcí
- funkce vracející pole
- funkce vracející funkci

Popis typu struktura

- **Syntaxe:**

```
struct značka { seznam popisů položek }
```

- **Popisy položek mají podobný tvar, jako deklarace**

- nesmí obsahovat paměťovou třídu
- položka nesmí být typu funkce nebo stejná struktura
- položka může být ukazatelem na funkci nebo stejnou strukturu

- **Příklad:**

```
struct osoba {  
    char jmeno[20];  
    char prijmeni[25];  
    int rok_narozeni;  
}; /* deklarace struktury */  
struct osoba Jan;  
/* deklarace proměnné typu struct osoba */
```

Popis typu struktura (pokračování)

- Značka (jméno, tag) struktury není v C identifikátorem typu

```
osoba Petr; /* v C je to chyba, v C++ není */
```

- Identifikátor typu lze pro strukturu zavést deklarací typu

```
typedef struct Complex {  
    float Real, Imag;  
} Complex;  
Complex a, b;                /* O.K. */
```

Další příklady struktur

- struktura pro binární strom nebo dvousměrně zřetězený seznam

```
struct tnode {  
    char    slovo[20];  
    int     pocet;  
    struct tnode *levy, *pravy;  
};
```

- anonymní struktura

```
struct { int prvni, druha; } p1[9], *p2;
```

- vzájemně odkazované struktury

```
struct S1;  
struct S2 { int Obsah; struct S1 *Dalsi; };  
struct S1 { long Obsah; struct S2 *Dalsi; };
```

Příklad: Tabulka zaměstnanců

```
#define MAXZAM 10

typedef struct osoba {
    unsigned int ID;
    char jmeno[25];
    char prijmeni[36];
    float plat;
} ZAM;                /* definice typu ZAM */

ZAM tab[MAXZAM];      /* tabulka zaměstnanců */
```

Popis výčtového typu

- Syntaxe:
`enum značka { seznam literálů }`
- Příklad:
`enum Color {Red, Blue, Green};`
- Literály jsou synonyma celočíselných hodnot
`/* Red = 0, Blue = 1, Green = 2 */`

Popis výčtového typu (pokračování)

- Literálu lze explicitně přiřadit hodnotu

```
enum Masky { Nula, Jedna, Dva, Ctyri=4, Osm=8 };  
/* Dva = 2, Ctyri = 4, Osm = 8 */
```

```
enum Sign { Minus = -1, Zero, Plus};  
/* Minus = -1, Zero = 0, Plus = 1 */
```

```
enum { E1, E2, E3 = 5, E4, E5 = E4 + 10, E6};  
/* E4 = 6, E5 = 16, E6 = 17 */
```

- Výčtový typ je kompatibilní s `int` i co do délky vnitřní reprezentace (pozor – v nových verzích C++ to nemusí platit)

```
enum Sign s; /* totéž co int s; */
```

The End